Core Building Blocks of Angular

In Angular, **Components**, **Data Binding**, **Directives**, and **Angular CLI** are the core building blocks. Let's understand them one by one.

1. Component

A Component is the basic building block of an Angular application. It controls a part of the user interface (UI) and contains:

- **HTML (Template)**
- **TypeScript (Logic)**
- **CSS (Styling)**

Example: `counter.ts`

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: '<h1>{{ title }}</h1>'
})
export class Counter {
  title = 'Counter Component';
}
```

Explanation

- `@Component` → Decorator that tells Angular this class is a component.
- `selector` → HTML tag used to display the component.

> **Usage:** `<app-counter></app-counter>`
> **Output:** Counter Component

Component Structure

```
Component
│
├── Template (HTML)
├── Class (TypeScript)
└── Style (CSS)
```

2. Data Binding

Data Binding connects data between the component (TypeScript) and the view (HTML). Angular supports four types of binding.

A. Interpolation

Displays component data in HTML.

```
// Component
name = "Mohanraj";

// Template
<h2>{{ name }}</h2>
```

Output: Mohanraj

B. Property Binding

Binds a component property to an HTML property.

```
// Component
imageUrl = "assets/logo.png";

// Template
<img [src]="imageUrl">
```

Here, `[src]` means Angular sets the src property from the component.

C. Event Binding

Binds an event from HTML to a method in TypeScript.

```
// Component
count = 0;
increment() {
  this.count++;
}

// Template
<button (click)="increment()">Increment</button>
```

When the button is clicked, the method executes.

D. Two-Way Binding

Updates data in both directions.

```
// Component
name = '';

// Template
<input [(ngModel)]="name">
<p>{{ name }}</p>
```

> **Shortcut:** `[(ngModel)]` combines Property Binding + Event Binding.

3. Directives

Directives are instructions that change the appearance or behavior of DOM elements.

A. Component Directive

Every component is a directive with a template.

```
@Component({...})
export class AppComponent {}
```

B. Structural Directives

These modify the DOM structure.

***ngIf:** Shows or hides elements.

```
<p *ngIf="isLoggedIn">Welcome</p>
```

If `isLoggedIn = true`, Output: Welcome. If false, nothing is displayed.

***ngFor:** Repeats elements.

```
// Component
movies = ["Leo", "Vikram", "Master"];

// Template
<li *ngFor="let movie of movies">{{ movie }}</li>
```

C. Attribute Directives

Change appearance or behavior.

ngClass: Adds or removes CSS classes.

```
<p [ngClass]="{'active': isActive}">Angular</p>
```

ngStyle: Applies styles dynamically.

```
<p [ngStyle]="{'color':'blue'}">Hello</p>
```

4. Angular CLI

Angular CLI (Command Line Interface) is a tool used to create, run, build, and manage Angular projects.

- **Check Version:** `ng version`
- **Create New Project:** `ng new myApp`
- **Run Application:** `ng serve` (Opens at http://localhost:4200)
- **Generate Component:** `ng generate component counter` or `ng g c counter`
- **Generate Service:** `ng g s employee`
- **Build Project:** `ng build`
- **Run Unit Tests:** `ng test`

Quick Summary

Concept	Purpose	Example
Component	UI building block	`@Component()`
Interpolation	Display data	`{{name}}`
Property Binding	TS → HTML	`[src]="url"`
Event Binding	HTML → TS	`(click)="save()"`
Two-Way Binding	Both directions	`[(ngModel)]`
*ngIf	Show/Hide element	`*ngIf="flag"`
*ngFor	Repeat elements	`*ngFor="let item of items"`
ngClass	Add CSS class	`[ngClass]="..."`
ngStyle	Apply styles	`[ngStyle]="..."`
Angular CLI	Angular command tool	`ng serve`

Simple Flow

These four topics are the foundation of Angular. Once you're comfortable with them, the next important concepts are Services, Dependency Injection (DI), Routing, Reactive Forms, and HTTP Client.